

Complete Guide to Python Dataclasses and Field Function

Table of Contents

1. [Introduction to Dataclasses](#)
 2. [Automatically Generated Methods](#)
 3. [Field Function Deep Dive](#)
 4. [Lambda vs Non-Lambda in default_factory](#)
 5. [Real-World Examples](#)
 6. [Advanced Patterns](#)
 7. [Summary Tables](#)
-

Introduction to Dataclasses

Dataclasses are a feature introduced in Python 3.7 that automatically generate common special methods for classes. They reduce boilerplate code and provide a clean way to create data containers.

Basic Syntax

```
from dataclasses import dataclass

@dataclass
class Person:
    name: str
    age: int = 25
    city: str = "Unknown"
```

What @dataclass Does

- Automatically generates `__init__`, `__repr__`, `__eq__`, and other special methods
 - Provides type hints and default values
 - Reduces boilerplate code significantly
 - Makes classes more readable and maintainable
-

Automatically Generated Methods

When you use `@dataclass`, Python automatically generates several special methods for you.

1. `__init__` Method

```
@dataclass
class Person:
    name: str
    age: int = 25
    city: str = "Unknown"

# This automatically creates:
def __init__(self, name: str, age: int = 25, city: str = "Unknown"):
    self.name = name
    self.age = age
    self.city = city
```

2. `__repr__` Method

```
# Automatically creates a readable string representation
person = Person("Alice", 30, "New York")
print(person) # Person(name='Alice', age=30, city='New York')
```

3. `__eq__` Method

```
# Automatically creates equality comparison
person1 = Person("Alice", 30)
person2 = Person("Alice", 30)
person3 = Person("Bob", 30)

print(person1 == person2) # True
print(person1 == person3) # False
```

4. `__hash__` Method (if frozen=True)

```
@dataclass(frozen=True)
class ImmutablePerson:
    name: str
    age: int

# Automatically creates hash method for immutable objects
person = ImmutablePerson("Alice", 30)
print(hash(person)) # Some hash value
```

5. `__post_init__` Method (if you define it)

```
@dataclass
class ValidatedPerson:
    name: str
    age: int

    def __post_init__(self):
        if self.age < 0:
            raise ValueError("Age cannot be negative")

# This runs automatically after __init__
```

Field Function Deep Dive

The `field` function is used to customize how individual fields behave in a dataclass. It's not actually a class, but a function that returns a `Field` object.

Field Parameters Explained

```

from dataclasses import dataclass, field

@dataclass
class Example:
    # Basic field with default value
    name: str = field(default="Unknown")

    # Field with factory function
    items: list = field(default_factory=list)

    # Field excluded from __init__
    computed_value: int = field(init=False)

    # Field excluded from __repr__
    secret_data: str = field(repr=False)

    # Field excluded from comparisons
    timestamp: float = field(compare=False)

    # Field excluded from hash
    cache: dict = field(hash=False)

    # Field with metadata
    version: str = field(metadata={"description": "API version"})

```

1. default Parameter

```

@dataclass
class Person:
    name: str = field(default="Anonymous")
    age: int = field(default=0)

# Equivalent to:
# name: str = "Anonymous"
# age: int = 0

```

2. default_factory Parameter

```

@dataclass
class Team:
    # Each instance gets its own list
    members: list = field(default_factory=list)

    # Each instance gets its own dict
    scores: dict = field(default_factory=dict)

    # Each instance gets its own set
    tags: set = field(default_factory=set)

# Usage:
team1 = Team()
team2 = Team()
team1.members.append("Alice")
print(team2.members) # [] - Empty list, not shared!

```

3. init Parameter

```
@dataclass
class User:
    username: str
    password: str = field(init=False) # Won't be in __init__

    def __post_init__(self):
        # Set password after initialization
        self.password = self._generate_password()

    def _generate_password(self):
        return f"pass_{self.username}_123"

# Usage:
user = User("john") # No password parameter needed
print(user.password) # "pass_john_123"
```

4. repr Parameter

```
@dataclass
class Account:
    username: str
    password: str = field(repr=False) # Hidden from string representati
    balance: float = 0.0

# Usage:
account = Account("john", "secret123", 100.0)
print(account) # Account(username='john', balance=0.0)
# Password is hidden!
```

5. compare Parameter

```
@dataclass
class Product:
    name: str
    price: float
    timestamp: float = field(compare=False) # Ignored in comparisons

# Usage:
product1 = Product("Book", 10.0, 1234567890.0)
product2 = Product("Book", 10.0, 9876543210.0)
print(product1 == product2) # True (timestamp ignored)
```

6. hash Parameter

```
@dataclass(frozen=True)
class CacheKey:
    user_id: int
    cache_data: dict = field(hash=False) # Won't affect hash

# Usage:
key1 = CacheKey(123, {"temp": "data"})
key2 = CacheKey(123, {"different": "data"})
print(hash(key1) == hash(key2)) # True (cache_data ignored)
```

7. metadata Parameter

```

@dataclass
class Configuration:
    timeout: int = field(
        default=30,
        metadata={
            "description": "Request timeout in seconds",
            "min_value": 1,
            "max_value": 300,
            "unit": "seconds"
        }
    )

# Usage:
config = Configuration()
field_info = Configuration.__dataclass_fields__['timeout']
print(field_info.metadata['description']) # "Request timeout in seconds"

```

Lambda vs Non-Lambda in default_factory

The Key Difference

Lambda Version:

```

rating_map: Dict[str, int] = field(default_factory=lambda: {
    'One': 1, 'Two': 2, 'Three': 3, 'Four': 4, 'Five': 5
})

```

Non-Lambda Version:

```

scraper: ScraperConfig = field(default_factory=ScraperConfig)

```

Why the Difference?

1. Lambda for Complex Literals

```

# Lambda is needed for complex literals (dicts, lists, etc.)
rating_map: Dict[str, int] = field(default_factory=lambda: {
    'One': 1, 'Two': 2, 'Three': 3, 'Four': 4, 'Five': 5
})

# Without lambda (WRONG):
rating_map: Dict[str, int] = field(default_factory={
    'One': 1, 'Two': 2, 'Three': 3, 'Four': 4, 'Five': 5
}) # This would be evaluated once at module level!

```

2. Non-Lambda for Classes

```

# Non-lambda for class constructors
scraper: ScraperConfig = field(default_factory=ScraperConfig)

# This calls ScraperConfig() each time a new instance is created

```

Examples Showing the Problem

Problem Without Lambda (Mutable Defaults)

```

@dataclass
class BadExample:
    items: list = field(default_factory=[]) # WRONG!

# This creates the SAME list for all instances!
obj1 = BadExample()
obj2 = BadExample()
obj1.items.append("item")
print(obj2.items) # ['item'] - Shared list!

```

Solution With Lambda

```
@dataclass
class GoodExample:
    items: list = field(default_factory=lambda: []) # CORRECT!

# This creates a NEW list for each instance
obj1 = GoodExample()
obj2 = GoodExample()
obj1.items.append("item")
print(obj2.items) # [] - Separate lists!
```

Alternative Solutions

```
@dataclass
class AlternativeExample:
    # Option 1: Lambda
    items1: list = field(default_factory=lambda: [])

    # Option 2: list constructor
    items2: list = field(default_factory=list)

    # Option 3: dict constructor
    config: dict = field(default_factory=dict)

    # Option 4: Custom function
    def create_default_dict():
        return {"key": "value"}

    custom: dict = field(default_factory=create_default_dict)
```

When to Use Each

Use Case	Lambda	Non-Lambda
Complex literals	lambda: {"key": "value"}	
Class constructors		MyClass
Simple constructors		list, dict, set
Custom functions		my_function
Mutable defaults	Required	Causes sharing

Real-World Examples

Example 1: Configuration System

```

@dataclass
class ScraperConfig:
    base_url: str = 'https://books.toscrape.com/'
    timeout: int = 10
    max_retries: int = 3

@dataclass
class AppConfig:
    max_books_to_show: int = 5
    log_file: str = 'logs.text'

@dataclass
class RatingConfig:
    # Lambda needed for complex dict literal
    rating_map: Dict[str, int] = field(default_factory=lambda: {
        'One': 1, 'Two': 2, 'Three': 3, 'Four': 4, 'Five': 5
    })

@dataclass
class Config:
    # Non-lambda for class constructors
    scraper: ScraperConfig = field(default_factory=ScraperConfig)
    app: AppConfig = field(default_factory=AppConfig)
    rating: RatingConfig = field(default_factory=RatingConfig)

```

Example 2: Data Processing

```

@dataclass
class DataProcessor:
    data: list = field(default_factory=list)
    cache: dict = field(default_factory=dict)
    processed_count: int = field(default=0, init=False)

    def __post_init__(self):
        self.processed_count = len(self.data)

    def add_item(self, item):
        self.data.append(item)
        self.processed_count += 1

```

Example 3: API Client

```

@dataclass
class APIClient:
    base_url: str
    headers: dict = field(default_factory=dict)
    session: requests.Session = field(init=False)

    def __post_init__(self):
        self.session = requests.Session()
        self.session.headers.update(self.headers)

```

Advanced Patterns

1. Computed Fields

```

@dataclass
class Rectangle:
    width: float
    height: float
    area: float = field(init=False)

    def __post_init__(self):
        self.area = self.width * self.height

```

2. Private Fields

```
@dataclass
class SecureUser:
    username: str
    _password_hash: str = field(init=False, repr=False)

    def __post_init__(self):
        self._password_hash = self._hash_password(self.username)
```

3. Conditional Fields

```
@dataclass
class ConditionalConfig:
    mode: str
    debug_info: dict = field(init=False)

    def __post_init__(self):
        if self.mode == "debug":
            self.debug_info = {"level": "verbose", "log_file": "debug.log"}
        else:
            self.debug_info = {}
```

4. Inheritance with Dataclasses

```
@dataclass
class Person:
    name: str
    age: int

@dataclass
class Employee(Person):
    salary: float
    department: str
```

5. Frozen Dataclasses

```
@dataclass(frozen=True)
class ImmutableConfig:
    api_key: str
    timeout: int

    # Cannot modify after creation
    # config.api_key = "new_key" # Raises FrozenInstanceError
```

Summary Tables

Table 1: Automatically Generated Methods by @dataclass

Method	When Generated	Purpose	Example
<code>__init__</code>	Always	Initialize object with field values	<code>Person("Alice", 30)</code>
<code>__repr__</code>	Always	String representation for debugging	<code>Person(name='Alice', age=30)</code>
<code>__eq__</code>	Always	Equality comparison	<code>person1 == person2</code>
<code>__hash__</code>	If <code>frozen=True</code>	Hash value for immutable objects	<code>hash(immutable_person)</code>
<code>__post_init__</code>	If you define it	Post-initialization processing	Validation, computed fields

Table 2: Field Function Parameters

Parameter	Purpose	Default	Example
default	Simple default value	MISSING	field(default=0)
default_factory	Function for default	MISSING	field(default_factory=list)
init	Include in __init__	True	field(init=False)
repr	Include in __repr__	True	field(repr=False)
compare	Include in comparisons	True	field(compare=False)
hash	Include in hash	None	field(hash=False)
metadata	Custom metadata	None	field(metadata={"unit": "kg"})

Table 3: When to Use Lambda vs Non-Lambda in default_factory

Use Case	Lambda	Non-Lambda	Example
Complex literals	Required	Not applicable	lambda: {"key": "value"}
Class constructors	Not needed	Use class name	MyClass
Simple constructors	Not needed	Use constructor	list, dict, set
Custom functions	Not needed	Use function name	my_function
Mutable defaults	Required	Causes sharing	lambda: []

Table 4: Common Field Patterns

Pattern	Purpose	Example
Mutable defaults	Prevent shared state	field(default_factory=list)
Computed fields	Derived values	field(init=False) + __post_init__
Private fields	Hide from repr/init	field(repr=False, init=False)
Metadata	Add custom info	field(metadata={"description": "..."})
Conditional fields	Runtime decisions	field(init=False) + __post_init__

Table 5: Dataclass Decorator Options

Option	Purpose	Example
frozen=True	Make immutable	@dataclass(frozen=True)
init=True	Generate __init__	@dataclass(init=True)
repr=True	Generate __repr__	@dataclass(repr=True)
eq=True	Generate __eq__	@dataclass(eq=True)
order=False	Generate ordering methods	@dataclass(order=True)
unsafe_hash=False	Generate __hash__	@dataclass(unsafe_hash=True)

Table 6: Best Practices

Practice	Do	Don't
Mutable defaults	Use <code>field(default_factory=...)</code>	Use <code>field(default=[])</code>
Complex literals	Use <code>lambda: {...}</code>	Use <code>{...}</code> directly
Class defaults	Use <code>field(default_factory=MyClass)</code>	Use <code>field(default=MyClass())</code>
Validation	Use <code>__post_init__</code>	Put logic in <code>__init__</code>
Private fields	Use <code>field(repr=False)</code>	Use <code>_</code> prefix only
Metadata	Use <code>field(metadata=...)</code>	Store in separate dict

Quick Reference

Basic Dataclass

```
@dataclass
class Person:
    name: str
    age: int = 25
```

With Field Customization

```
@dataclass
class Config:
    items: list = field(default_factory=list)
    secret: str = field(repr=False)
    computed: int = field(init=False)
```

With Validation

```
@dataclass
class Validated:
    value: int

    def __post_init__(self):
        if self.value < 0:
            raise ValueError("Value must be positive")
```

Immutable Dataclass

```
@dataclass(frozen=True)
class Immutable:
    value: str
```

This comprehensive guide covers all aspects of Python dataclasses and the field function, providing practical examples and best practices for effective usage.